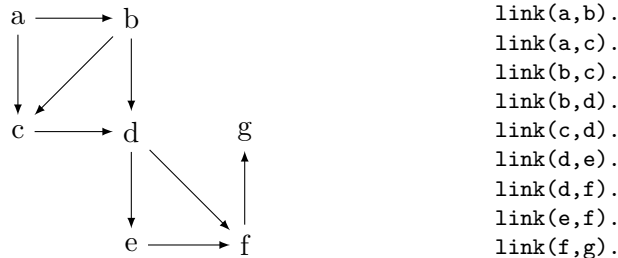


Homework 5

You may submit your solution to this homework assignment in the text window in iCollege, as a `.txt` file, or in some other commonly used format, *e.g.*, `.pdf`, `.docx`, *etc.*

1. Consider the (directed) network on the left-hand side, below. We could represent this network with the set of Prolog statements on the right-hand side, below.



Now, given this network, we say that there is a *path* from a node *X* to a node *Y* if we can get from *X* to *Y* via a series of links. For example, in this network, there is a path from *a* to *d*, and a path from *c* to *f*, etc.

- (a) Formulate the appropriate Prolog rule `path(X,Y)` which is true if there is a `path` from *X* to *Y* as described above — note that this rule will be recursive. Test this rule out on the above network, to see if it is working correctly.

Once it is working correctly, you will note that, *e.g.*, the query `path(a,e).` will give `true` multiple times. This means something, actually.

- (b) How many times is `true` given for the query `path(a,e).`, and what does this mean?

2. Remember the `init` function of Haskell? It can be defined in Haskell as:

```
init :: [a] -> [a]
init [] = []
init (x:xs) = x : init xs
```

Formulate the appropriate Prolog rule `init(Xs,Ys)` which is true if *Ys* is the initial part of *Xs*. That is, the query `init([1,2,3,4,5],[1,2,3,4]).` would evaluate to `true`, or the query `init([1,2,3,4,5],Ys).` would give *Ys* = `[1,2,3,4]`.

Hint: this rule will be recursive, and will have a corresponding base case and recursive case, analogous to the base case and recursive case, respectively, of the above Haskell function — the main difference being that the Prolog rule `init(Xs,Ys)` is a predicate, not a function.

3. Remember the `zip` function of Haskell? It can be defined in Haskell as:

```
zip :: [a] -> [b] -> [(a,b)]
zip _ [] = []
zip [] _ = []
zip (x:xs) (y:ys) = (x,y) : zip xs ys
```

Formulate the appropriate Prolog rule `zip(Xs,Ys,Zs)` which is true if *Zs* is the result of “zipping” together *Xs* and *Ys*. That is, the query `zip([1,2,3],[4,5],[(1,4),(2,5)]).` would evaluate to `true`, or the query `zip([1,2],[4,5,6],Zs).` would give *Zs* = `[(1,4),(2,5)]`.

Hint: again, this rule will have two base cases and a recursive case analogous to the three cases of the above Haskell function.